

STM32 Ethernet Tutorial

Using CGI & HTTP Server

LED Control over Ethernet with LwIP & STM32CubeMX

Field	Detail
Microcontroller	STM32F767ZI (Nucleo-144)
IDE	STM32CubeMX v6.6.1
Firmware Pack	STM32Cube FW_F7 V1.17.0
Protocol Stack	LwIP TCP/IP (HTTP + CGI)

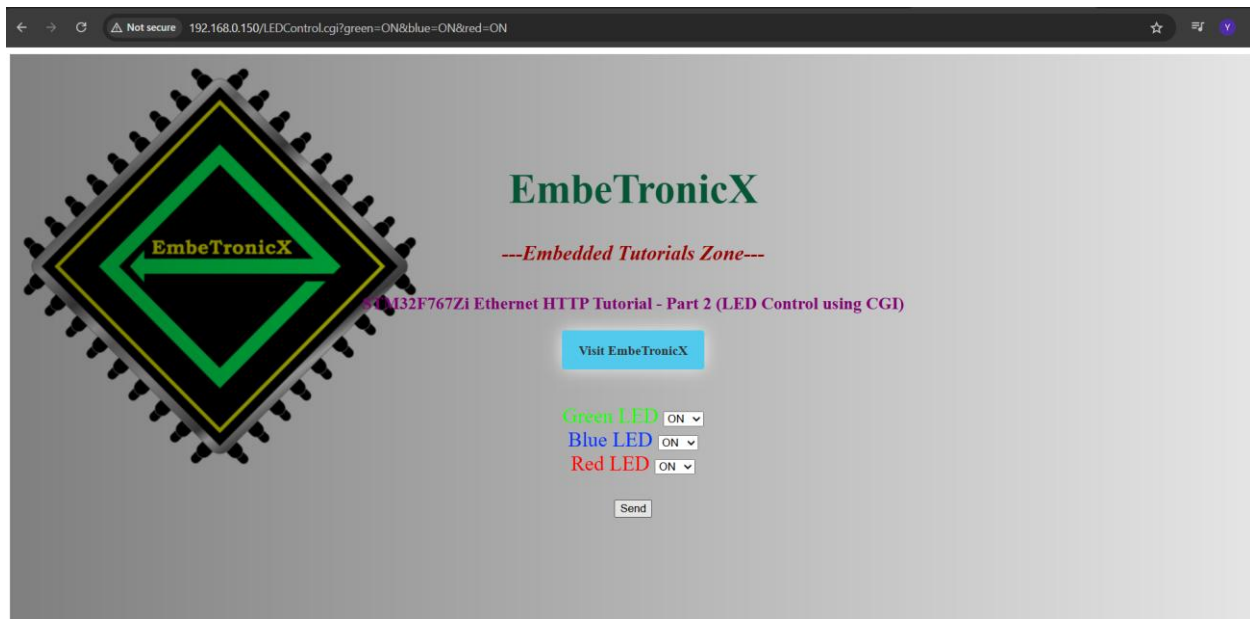


Figure 1: LED Control via CGI HTTP Server

1. Project Overview

This document provides complete technical documentation for the STM32 Ethernet HTTP Server project using Common Gateway Interface (CGI). The project demonstrates how to set up a lightweight web server on an STM32F767ZI Nucleo-144 board using the LwIP TCP/IP stack, enabling a browser-based interface to control onboard LEDs over an Ethernet connection.

CGI is the mechanism that is part of the Hypertext Transport Protocol (**HTTP**). The Common Gateway Interface (CGI) is the standard process that uses a set of rules to propagate the user's request to the web server and respond to the user through the web interface. As CGI is a standard interface, the communication between the web server and the client's web browser follows specified rules.

This is Part 2 of the STM32 Ethernet Tutorial series, configured using STM32CubeMX v6.6.1 with STM32Cube FW_F7 V1.17.0.

2. Hardware Requirements

Component	Specification / Model	Notes
Development Board	STM32F767ZI Nucleo-144	Onboard Ethernet PHY
Green LED	GPIO PB0	Controlled via CGI
Blue LED	GPIO PB7	Controlled via CGI
Red LED	GPIO PB14	Controlled via CGI
Network Interface	RJ45 LAN Cable	Connected to Router
Router / Switch	Any standard router	IP assignment & LAN access
Host PC / Mobile	Browser-capable device	Connected to same router

3. Software & Tools

- STM32CubeMX v6.6.1 — Project configuration and code generation
- STM32Cube FW_F7 V1.17.0 — HAL firmware library for STM32F7 series
- LwIP TCP/IP Stack — Integrated via STM32CubeMX Middleware
- Perl — Required to run the makefsdata.pl script (<https://www.perl.org/get.html>)
- STM32CubeIDE / Keil / IAR — For compiling and flashing firmware
- Web Browser (Chrome, Firefox, Edge) — For accessing the HTTP server

4. STM32CubeMX Configuration

4.1 Enabling CGI in LwIP Middleware

Navigate to Middleware → LWIP → HTTPD and enable the following option:

- LWIP_HTTPD_CGI — Enable HTTP CGI (Old Style)

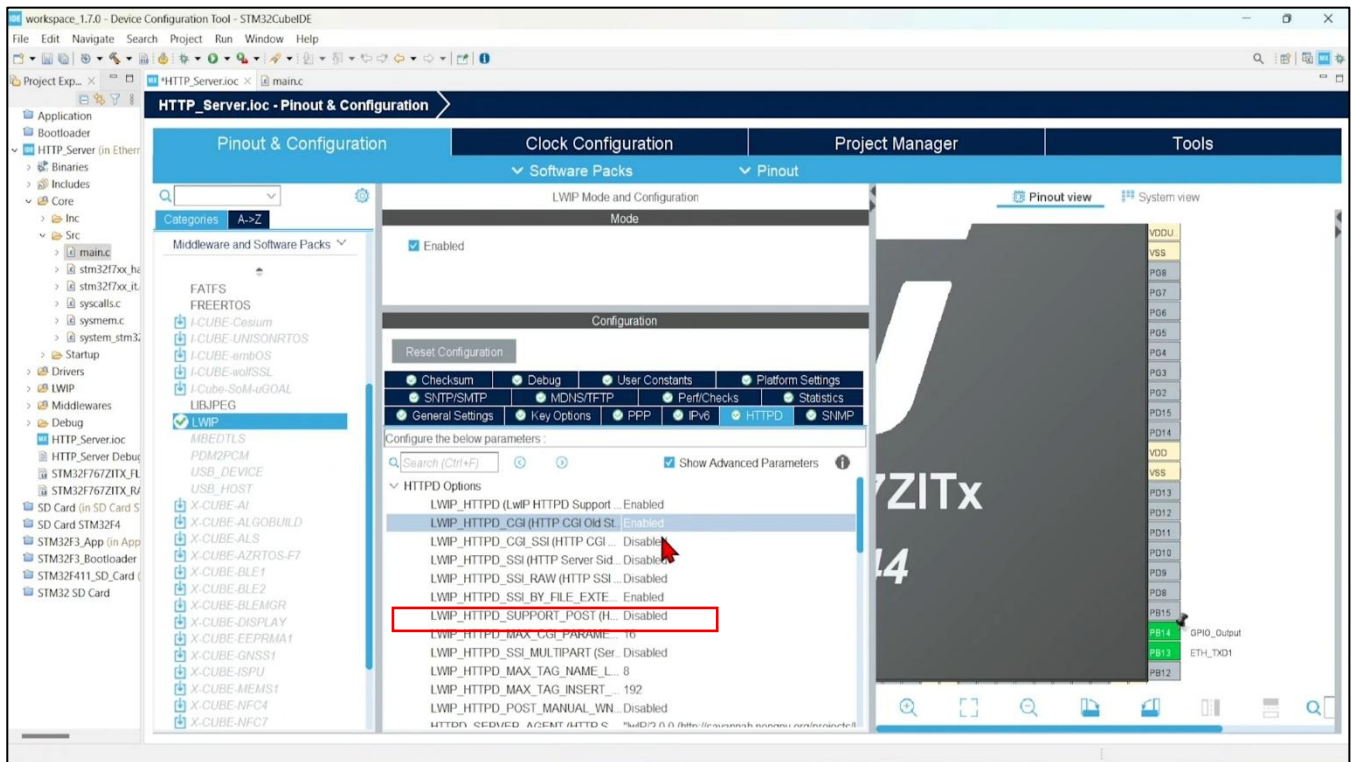


Figure 2: LWIP_HTTPD_CGI — Enable HTTP CGI

This enables CGI support within the LwIP HTTP server, allowing the firmware to register CGI handler callbacks triggered by HTTP GET requests from the browser.

4.2 GPIO Configuration for Onboard LEDs

Configure the following GPIO pins as Output Push-Pull to control the three onboard LEDs:

GPIO Pin	LED Color	Direction	Label
PB0	Green	GPIO Output	LD1
PB7	Blue	GPIO Output	LD2
PB14	Red	GPIO Output	LD3

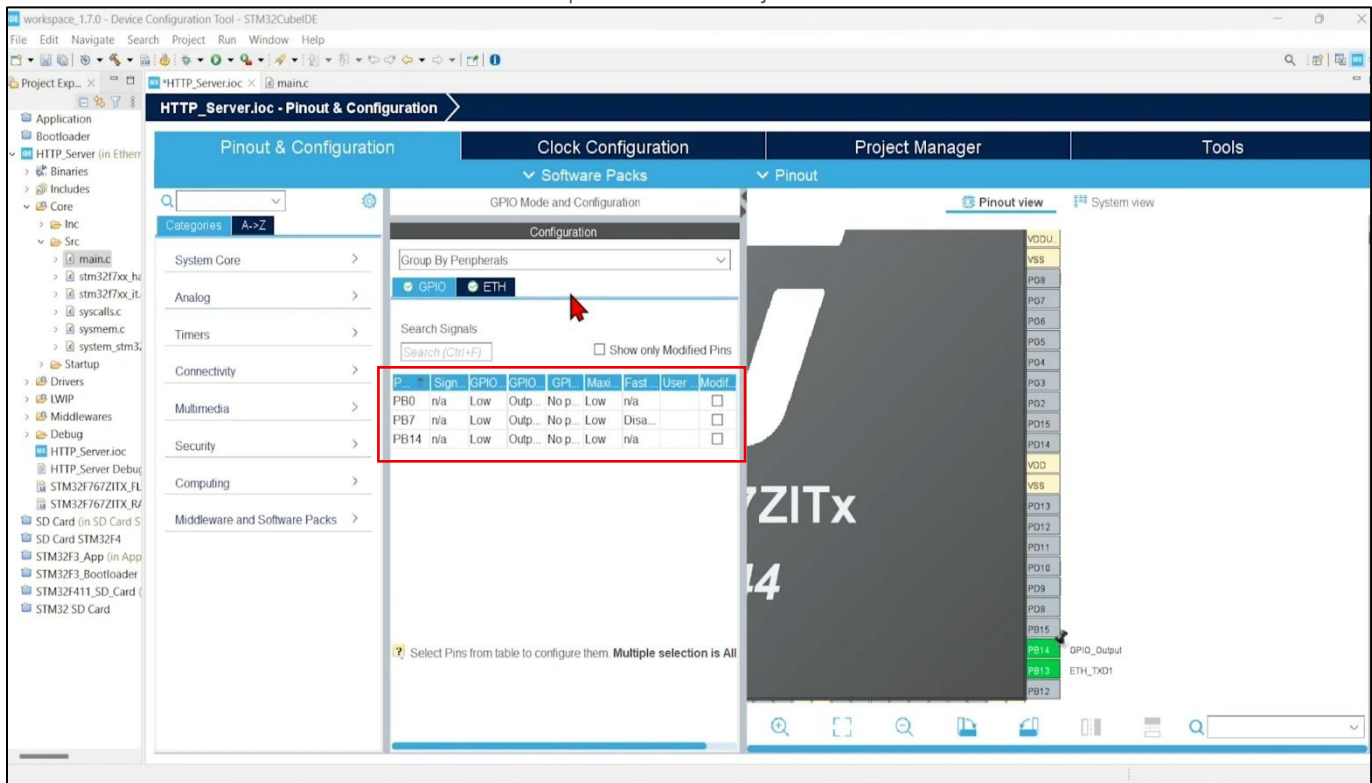


Figure 3: GPIO Configuration - PBO, PB7, PB14 set as Output

After enabling the pins, save the .ioc file. STM32CubeMX will automatically generate the initialization code for these GPIOs.

5. HTML Source Code

The HTML file below serves as the web interface hosted on the STM32. It contains a CGI form with three dropdown parameters (green, blue, red), each accepting ON or OFF values. On submit, the browser sends an HTTP GET request to LEDControl.cgi with the selected values.

5.1 File: index.html

```
<html>
<head>
<style>
  body { font-size:25px; color: #065535; }
  .green { color: #13FF00; }
  .blue { color: #002EFF; }
  .red { color: #FF0000; }
  form { padding:40px 20px; text-align:center; }
</style>
</head>
<body>
<div class="wd">
  <h1>EmbeTronicX</h1>
  <h4><i>---Embedded Tutorials Zone---</i></h4>
  <h5>STM32F767Zi Ethernet HTTP Tutorial - Part 2</h5>

  <FORM ACTION="LEDControl.cgi" METHOD="GET">
```

```
<div class="green">
  Green LED <SELECT NAME="green" SIZE=1>
    <OPTION SELECTED>ON <OPTION>OFF
  </SELECT>
</div>
<div class="blue">
  Blue LED <SELECT NAME="blue" SIZE=1>
    <OPTION SELECTED>ON <OPTION>OFF
  </SELECT>
</div>
<div class="red">
  Red LED <SELECT NAME="red" SIZE=1>
    <OPTION SELECTED>ON <OPTION>OFF
  </SELECT>
</div>
<br>
<input value="Send" type="submit">
</FORM>
</div>
</body>
</html>
```

6. STM32 Firmware Code (main.c)

All additions are placed inside the designated USER CODE sections so they survive STM32CubeMX code re-generation.

6.1 Include Headers [USER CODE BEGIN Includes]

```
#include <string.h>
#include "lwip/apps/httpd.h"
```

6.2 Global Variables [USER CODE BEGIN PV]

```
int GreenLED = 0;
int BlueLED  = 0;
int RedLED   = 0;
```

6.3 Function Prototype [USER CODE BEGIN PFP]

```
const char *LedControlCgiHandler(int iIndex, int iNumParams,
                                  char *pcParam[], char *pcValue[]);
```

6.4 CGI Structure [USER CODE BEGIN 1]

```
tCGI LED_CGI = {"/LEDControl.cgi", LedControlCgiHandler};
```

6.5 Initialization [USER CODE BEGIN 2]

```
httpd_init();
http_set_cgi_handlers(&LED_CGI, 1);
```

`http_set_cgi_handlers()` registers the `LedControlCgiHandler` callback with the LwIP HTTP server.

6.6 Main Loop [while(1)]

```
while (1)
{
    MX_LWIP_Process();

    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GreenLED); // Green LED
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_7, BlueLED);  // Blue LED
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_14, RedLED);   // Red LED
}
```

6.7 CGI Handler Function [USER CODE BEGIN 4]

```
const char *LedControlCgiHandler(int index, int numParams,
                                  char *pcParam[], char *pcValue[])
{
    if (index == 0)
    {
        for (int i = 0; i < numParams; i++)
        {
            if (strcmp(pcParam[i], "green") == 0)
                GreenLED = (strcmp(pcValue[i], "ON") == 0) ? 1 : 0;
            else if (strcmp(pcParam[i], "blue") == 0)
                BlueLED = (strcmp(pcValue[i], "ON") == 0) ? 1 : 0;
            else if (strcmp(pcParam[i], "red") == 0)
                RedLED = (strcmp(pcValue[i], "ON") == 0) ? 1 : 0;
        }
    }
    return "/index.html";
}
```

6.8 CGI Handler — Parameter Reference

Argument	Type	Description
index	int	Index of the CGI form. Always 0 for a single form.
numParams	int	Number of parameters received (3 in this project: green, blue, red).
pcParam[]	char*	Array of parameter name strings (e.g., "green", "blue", "red").
pcValue[]	char*	Array of parameter values corresponding to pcParam ("ON" or "OFF").

7. Filesystem Image Generation (makefsdata)

The LwIP HTTP server cannot serve raw HTML files directly. The HTML content must be converted to C arrays compiled into the firmware using the `makefsdata.pl` Perl script.

7.1 Prerequisites

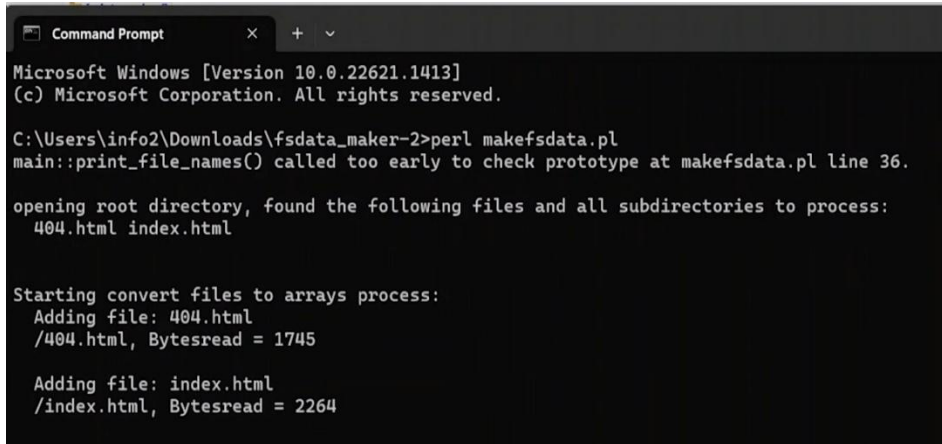
- Install Perl from <https://www.perl.org/get.html>
- Download and extract the `makefsdata` tool ZIP archive
- Place `index.html` and `404.html` in the root folder of the extracted directory

7.2 Running makefsdata

Open a terminal in the extracted directory and run:

```
| perl makefsdata.pl
```

Expected output:



```
Microsoft Windows [Version 10.0.22621.1413]
(c) Microsoft Corporation. All rights reserved.

C:\Users\info2\Downloads\fsdata_maker-2>perl makefsdata.pl
main::print_file_names() called too early to check prototype at makefsdata.pl line 36.

opening root directory, found the following files and all subdirectories to process:
 404.html index.html

Starting convert files to arrays process:
Adding file: 404.html
 /404.html, Bytesread = 1745

Adding file: index.html
 /index.html, Bytesread = 2264
```

7.3 Adding Generated Files to the Project

Copy fsdata_custom.c and fsdata.c into the following directory:

```
| Middlewares > Third_Party > LwIP > Src > apps > http
```

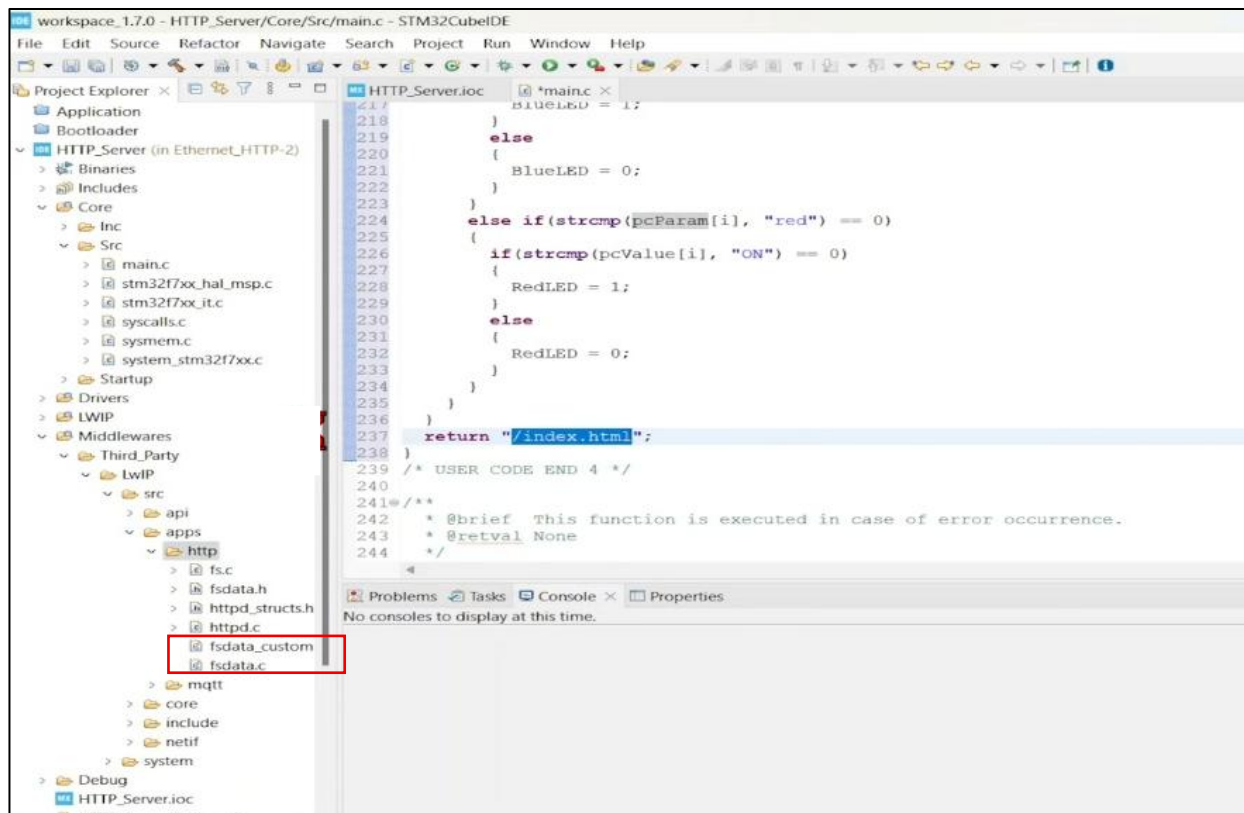


Figure 4: fsdata_custom.c and fsdata.c added to http

7.4 Excluding Files from Build

To avoid duplicate symbol errors, exclude both files from the build:

1. Select fsdata_custom.c and fsdata.c in the Project Explorer.
2. Right-click → Resource Configurations → Exclude from Build.
3. Select both Debug and Release configurations, then click OK.

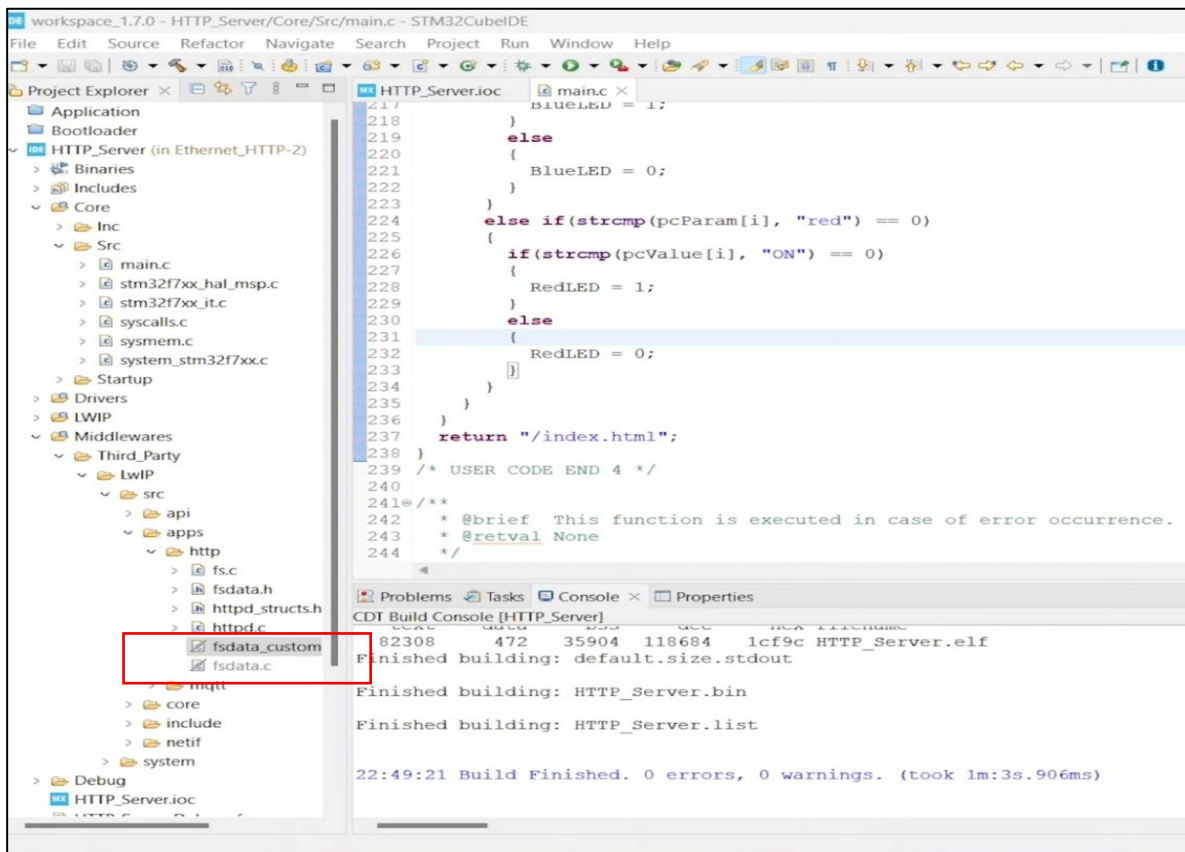


Figure 5: Exclude from Build — Debug and Release selected

8. Hardware Connection

- Connect the RJ45 LAN cable from the STM32F767ZI Nucleo-144 to the router.
- Ensure your PC or mobile device is on the same router (Wi-Fi or LAN).
- Verify the STM32 board's assigned IP address (192.168.0.150).

9. Build, Flash & Test

9.1 Build the Project

Build the project in STM32CubeIDE. Confirm there are no compilation errors before flashing.

9.2 Flash the Firmware

Flash the .bin or .elf file to the STM32F767ZI using the onboard ST-LINK debugger.

9.3 Accessing the Web Interface

After flashing, restart the board. Open a browser and navigate to:

| `http://192.168.0.150`

The LED control page will appear. Select ON or OFF for each LED and click Send. The onboard LEDs will respond immediately.

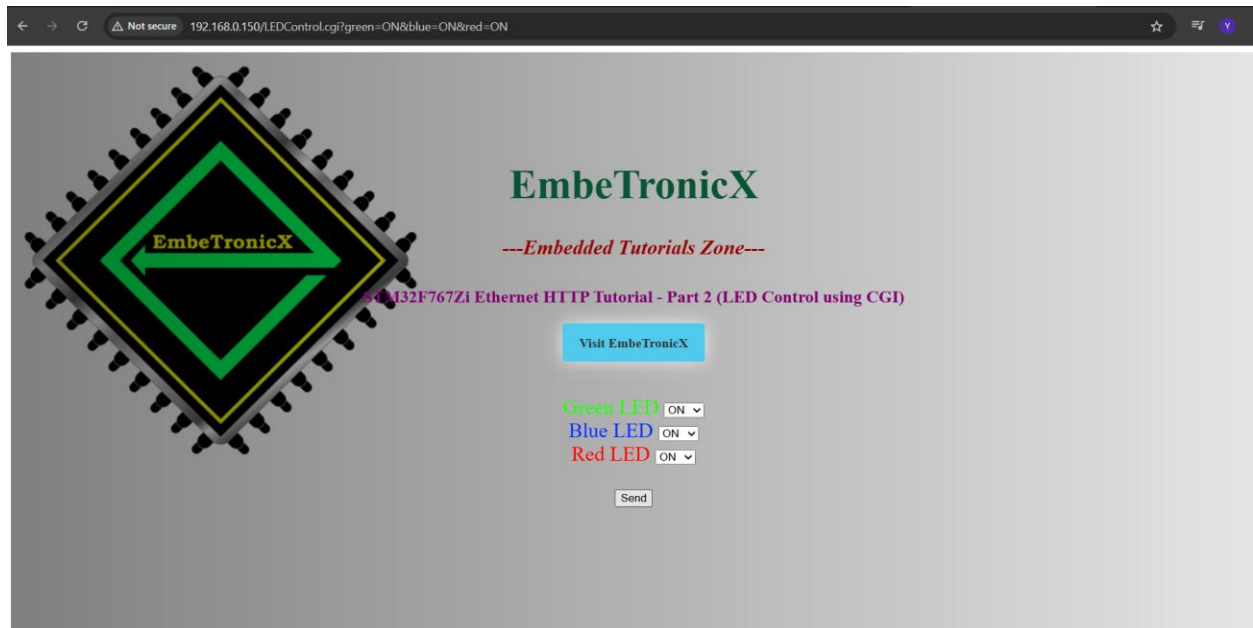


Figure 6: LED Control Page at 192.168.0.150

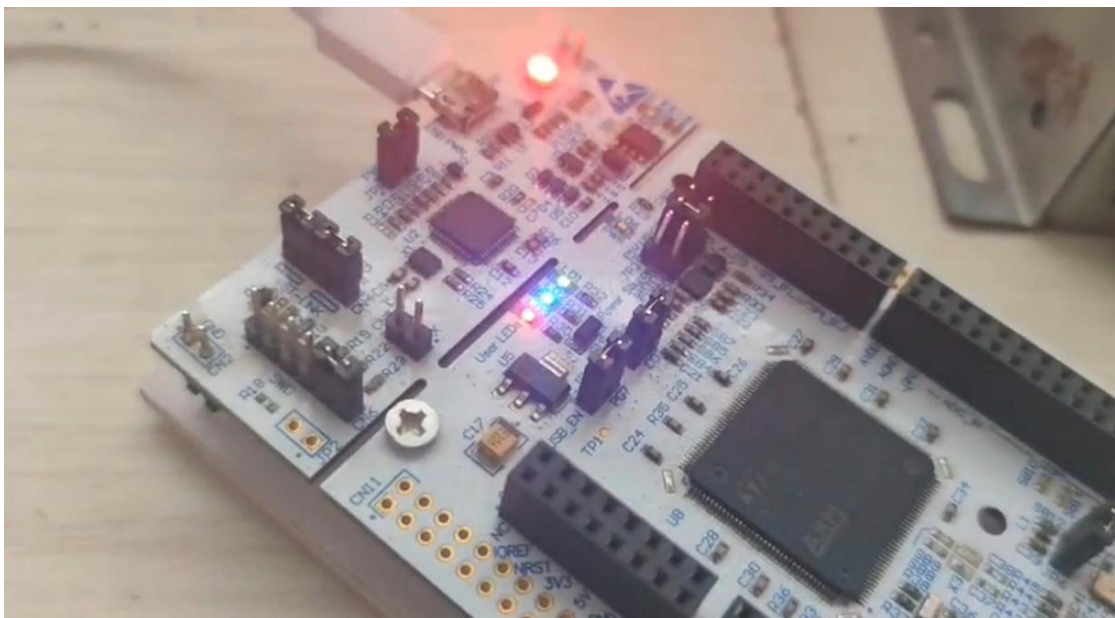


Figure 7: STM32 Nucleo Board with LEDs ON after form submission

10. How It Works — Data Flow

Step	Description
1	Browser navigates to 192.168.0.150. LwIP serves index.html from the embedded filesystem.
2	User selects LED states and clicks Send.
3	Browser sends HTTP GET to LEDControl.cgi with query params (e.g., ?green=ON&blue=OFF&red=ON).
4	LwIP matches the URL to the registered CGI handler via http_set_cgi_handlers().
5	LedControlCgiHandler() is called. Parameters are parsed; GreenLED, BlueLED, RedLED variables updated.
6	Handler returns "/index.html"; browser reloads the control page.
7	In main loop, HAL_GPIO_WritePin() applies the variable states to PB0, PB7, PB14.

11. Troubleshooting

Issue	Possible Solution
Browser cannot reach 192.168.0.150	Check both devices are on the same subnet. Verify LAN cable and IP address in code.
Page loads but LEDs do not respond	Ensure httpd_init() and http_set_cgi_handlers() are called. Verify GPIO pins are set as outputs.
Build errors after adding fsdata files	Confirm both files are excluded from Debug and Release builds.
makefsdata.pl produces no output	Ensure Perl is installed and in the system PATH. Run from the correct directory.
404 error when accessing the page	Verify index.html is in the correct root folder and the generated C files are included.

12. References

- STM32F767ZI Datasheet — STMicroelectronics
 - STM32CubeMX User Manual (UM1785) — STMicroelectronics
 - LwIP TCP/IP Stack Documentation — <https://savannah.nongnu.org/projects/lwip/>
 - STM32Cube FW_F7 V1.17.0 Release Notes — STMicroelectronics
 - EmbeTronicX STM32 Ethernet Tutorials — <https://embetronicx.com/>
 - Perl Official Website — <https://www.perl.org/get.html>
-